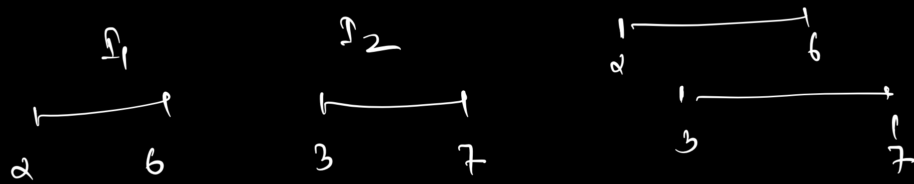
Intervals



↓

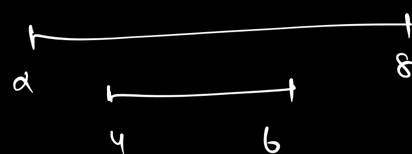
overlapping

↓

merge

[2 - 7]

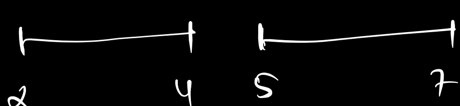
Q1 (2,8) (4,6)

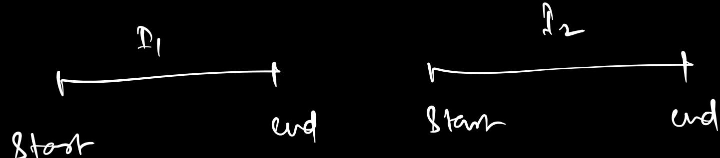


→ (2,8)

Q2. (2,4) (5,7)

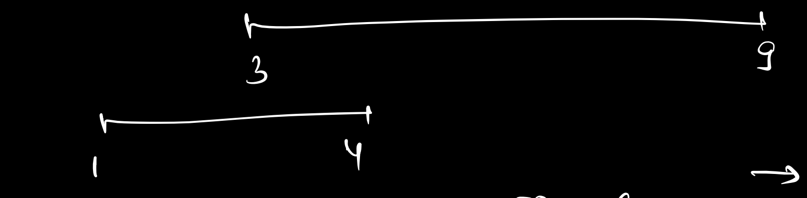
→ non overlapping





if I_2.start > I_1.end → no overlapping

Q3   (3, 9) (1, 4)



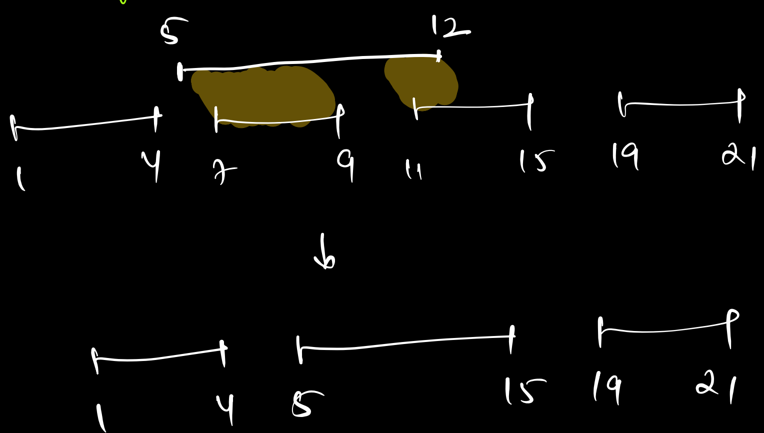
⇒ (1, 9)

Q1. Merge Intervals

Given a sorted array of non-overlapping interval, and given another interval, add this interval into the array and update the list of non-overlapping intervals.

ex ⇒

arr ⇒ [(1, 4)
(7, 9)
(11, 15)
(19, 21)]



I ⇒ (5, 12)

O/P

O/P ⇒ [(1, 4), (5, 15), (19, 21)]

I

ans

arr ⇒ [1,3] — ~~[12,22]~~ → [1,3]

[4,7] — ~~[12,22]~~ → [4,7]

[10,14] ↔ [12,22] → [10,22] →

[16,19] ↔ [10 22] → [10,22]

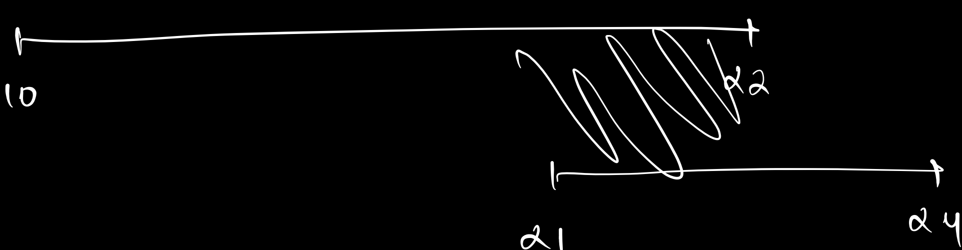
[21,24] → [10,22] → [10,24]

[27,30] → [10,24] → [10,24]

[32,35]

[27,30]

[32,35]



I

ans

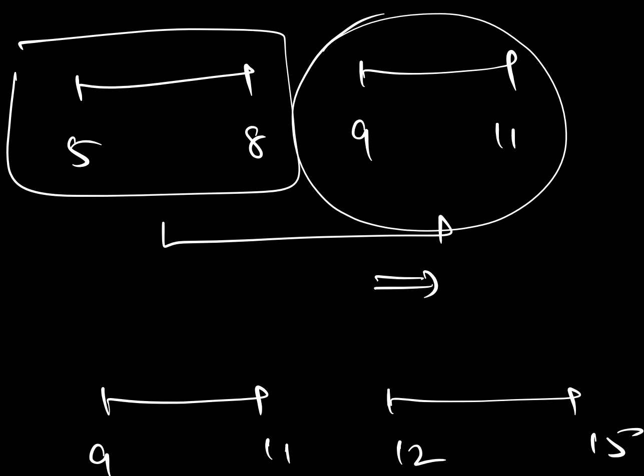
arr => [5,8] ~~[9,11]~~ → [5,8]

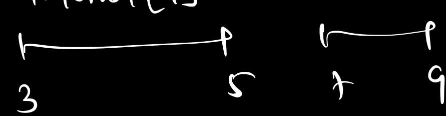
[9,11]

[12,15] ~~[9,11]~~ → [12,15]

[17,19] → [17,19]

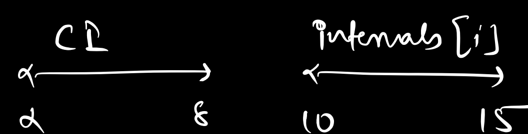
[23,25] → [23,25]





intervals[i].end

< CI.start



CI.end < intervals[i].start

Intervals [N][2]   Interval - start & end

CI => [start, end]

```
for( i=0; i<N; i++){
    if( intervals[i].end < CI.start) {
            ans.insert( intervals[i])
    }
    else if( intervals[i]. start > CI.end ) {
            ans.insert( CI)
            while( i < N) {
                    ans.insert( intervals[i])
                    i++
            }
            return ans;
    }
    else {
        CI.start = min( interval[i].start, CI.start)
        CI.end = max( interval[i].end, CI.end )
    }
}   ans.insert (CI)
    return ans.
}
```

2 10

1 3 → 1, 10

5 7 → 1, 10

9 11 → 1, 11

ans

1, 11

Q2. First missing natural no. / positive no.

Given an array of size N. Find the first missing +ve integer / natural no. in the array

ex ⇒ { 3, -2, 1, 2, 7 }

1 2 3 ~~4~~ → O/P = 4.

ex ⇒ { -9 2 6 4 -8 1 3 }

1 2 3 4 ~~5~~ → O/P = 5.

Q ⇒ { 1, 2, 5, 6, 4, 3 } ⇒ O/P = 7

1 2 3 4 5 6

(N ⇒ 1 → N)

O/P ⇒ N+1

Q ⇒ $\{1, 0, -5, -6, 4, 2\}$ ⇒ 6

$1 \to 6$

1 2 $\cancel{3}$ → O/P = 3.

N ⇒ $(1 \to N)$ → return missing

$(1 \to N)$ all present → $\underline{\underline{N+1}}$

Brute force

N → linear search (1 to N)

① $\boxed{\begin{array}{l} TC \Rightarrow O(N^2) \\ SC \Rightarrow O(1) \end{array}}$

Optimise

② extra space HashMap | HashSet

ex ⇒ $\{-1, 0, 3, 6, 9, 4, 1\}$ ⇒ $N = 7$.

boolean array [8] ⇒

~~F~~ T	~~F~~ T	F	~~F~~ T	~~F~~ T	F	~~F~~ T	F
0	1	2	3	4	5	6	7

present

```
for( i=1; i<=N; i++){
        if( present[i] == false)
                return i
}
    return N+1.
```

$$\boxed{\begin{array}{l} TC \Rightarrow O(N) \\ SC \Rightarrow O(N) \end{array}}$$

(3) array ⇒ {-1, 0, 3, 6, 9, 3, 4, 1}

sort ⇒ {-1, 0, 1, 3, 4, 6, 9}

O(N log N)

N = 7 [1 to 7]

[-1 0 1 3 4 6 9]

return c+1 → 2

count = 1

$$\boxed{\begin{array}{l} TC \Rightarrow O(N\log N) \\ SC \Rightarrow O(\;) \rightarrow \text{depends on algo.} \end{array}}$$

(iv) arr[N] → 1 → N

↓

indexes

0 to N-1

→ inc natural no.

(0 to N-1) to (1 to N)

(3) →

arr[6] ⇒ { 1 3 2 6 4 5 }

0 1 2 3 4 5

{ 1 2 3 4 5 6 }

0 1 2 3 4 5

arr[i] = i+1

element	index
x	$x-1$
arr[i]	arr[i]-1

ex => arr =>

N = 7

1 to 7

0	1	2	3	4	5	6
-1	0	3	6	9	1	4
x	x	✓				

0	1	2	3	4	5	6
-1	0	3	1	9	6	4
x	x	✓				

0	1	2	3	4	5	6
1	0	3	-1	9	6	4

0	1	2	3	4	5	6
1	0	3	4	9	6	-1

verify

0	1	2	3	4	5	6
1	0	3	4	9	6	-1

arr[i] = i+1

i+1 → O/P = 2

eg ⇒

$N = 6$

1 to 6

0	1	2	3	4	5
2	4	1	2	-10	7

0	1	2	3	4	5
4	2	1	2	-10	7

0	1	2	3	4	5
2	2	1	4	-10	7

0	1	2	3	4	5
1	2	2	4	-10	7

verify ⇒

O/p = i+1 = 3

```
int i=0
while ( i<N) {
    if( arr[i] >= 1 && arr[i] <= N) {
        correctIdx = arr[i]-1
        if( arr[correctIdx] != arr[i] ) {
            swap ( arr[correctIdx], arr[i])
}
```

if (false)
i++

else { i++

}
else { i++

}

for (i = 0 → N)

verify

$\boxed{\begin{array}{l} TC \Rightarrow O(N) \\ SC \Rightarrow O(1) \end{array}}$

if not allowed to modify i/p arr

$\boxed{\begin{array}{l} TC \Rightarrow O(N) \\ SC \Rightarrow O(N) \end{array}}$ ⇒ (II) / (IV)

(MAANG)

Q3. Max^m Absolute Difference

Given an array of size N, return max value

$|arr[i] - arr[j]| + |i - j|$

ex ⇒

0	1	2	3	4
2	5	4	1	6

$|6 - 1| + |4 - 3| = 5 + 1 = 6.$

$|6 - 2| + |4 - 0| = 4 + 4 = 8$ → O/P

Brute force

all possible combⁿ (i, j)

$$\boxed{\begin{aligned} TC &\Rightarrow O(N^2) \\ SC &\Rightarrow O(1) \end{aligned}}$$

$$|x| = -x \qquad x < 0$$

$$= x \qquad x \geq 0$$

$$|arr[i] - arr[j]| + |i - j|$$

(I) $arr[i] > arr[j]$ && $i > j$.

$$|arr[i] - arr[j]| + |i - j|$$

$$= arr[i] - arr[j] + i - j$$

$$= (arr[i] + i) - (arr[j] + j) \quad \text{— (I)}$$

(II) $arr[i] > arr[j]$ && $i < j$.

$$|arr[i] - arr[j]| + |i - j|$$

$$= arr[i] - arr[j] - (i - j)$$

$= arr[i] - arr[j] - i + j$

$= (arr[i] - i) - (arr[j] - j)$ —— (II)

(III) $arr[i] < arr[j]$ && $i > j$

$|arr[i] - arr[j]| + |i - j|$

$= -(arr[i] - arr[j]) + (i - j)$

$= -arr[i] + i + arr[j] - j$

$= (arr[j] - j) - (arr[i] - i)$ —— (III)

IV) $arr[i] < arr[j]$ && $i < j$

$|arr[i] - arr[j]| + |i - j|$

$= -(arr[i] - arr[j]) - (i - j)$

$= arr[j] - arr[i] + j - i$

$= (arr[j] + j) - (arr[i] + i)$ —— (IV)

$(arr[i]+i) - (arr[j]+j) \longrightarrow \text{(I)}$

$(arr[i]-i) - (arr[j]-j) \longrightarrow \text{(II)}$

$(arr[j]-j) - (arr[i]-i) \longrightarrow \text{(III)}$

$(arr[j]+j) - (arr[i]+i) \longrightarrow \text{(IV)}$

$$ans = \max\left(\underline{(arr[i]+i) - (arr[j]+j)},\ (arr[i]-i) - (arr[j]-j) \right)$$

$\underline{(arr[i]+i)} - (arr[j]+j) \Rightarrow 6.$

$\downarrow$ maxm $\qquad \downarrow$ minm

$\uparrow \qquad \uparrow$

$(arr[i]-i) - (arr[j]-j) \Rightarrow \quad 4-(-2)=6$

$\longrightarrow 6$

	0	1	2	3	4
	2	5	4	1	4
arr[i] + i $\Rightarrow$	2	6	6	4	8
arr[i] − i $\Rightarrow$	2	4	2	−2	0

maxm = 4

minm = −2

(i) maxm value arr[i] + i → maxSum
minm value arr[i] + i → minSum

op (i) → maxSum − minSum

(ii) maxm value arr[i] − i → maxDiff
minm value arr[i] − i → minDiff

op (ii) → maxDiff − minDiff

ans ⇒ max (op1, op2)

$\boxed{\begin{array}{l} TC \Rightarrow O(N) \\ SC \Rightarrow O(1) \end{array}}$

	0	1	2	3	4		
	2	5	4	1	6		
arr[i] + i ⇒	2	6	6	4	10	→	= 10 − 2 = 8
arr[i] − i ⇒	2	4	2	−2	2	→	4 − (−2) = 6

8